



NAND-GATE BASED HDL IMPLEMENTATION

Implementation of NOT, AND, OR, XOR, 2-Way MUX and DMUX Gates Using Only
NAND Gates

Project Report

This project demonstrates how common digital logic gates can be constructed using a single universal gate: the NAND gate. Each circuit is described in HDL (Hardware Description Language), along with its working principle and truth table.

Project Components	Included
Basic Gates	NOT, AND, OR, XOR
Routing Gates	2-Way MUX, 2-Way DMUX
Implementation Basis	NAND gate only
Description	Logic operation and HDL construction
Verification	Truth tables for all gates

Note: The HDL syntax used in this report follows the NAND2Tetris-style HDL convention, where a NAND primitive is available and all other gates are built from it.

1. Introduction and NAND as a Universal Gate

A NAND gate is called a **universal logic gate** because any Boolean function can be implemented using NAND gates alone. This makes NAND particularly useful for studying digital logic construction and hardware abstraction.

A NAND operation is the negation of AND:

$$\text{NAND}(A, B) = \text{NOT}(A \text{ AND } B)$$

The NAND truth table is:

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Basic construction techniques

NOT: Connect the same input to both NAND inputs. This produces NOT A.

AND: First NAND the inputs, then NAND the result with itself to invert it.

OR: Invert both inputs using NAND-as-NOT and then NAND those inverted signals.

XOR: Use a standard four-NAND construction to produce A XOR B.

MUX/DMUX: Use NAND gates to generate the selected and deselected paths.

2. NOT Gate Using NAND Gate

A NOT gate has one input and one output. Its output is the complement of the input. A NAND gate becomes a NOT gate when the same signal is connected to both NAND inputs.

Boolean expression: $Y = \text{NOT}(A)$

NAND implementation: $Y = A \text{ NAND } A$

HDL Code

```
CHIP Not {  
  IN in;  
  OUT out;  
  
  PARTS:  
    Nand(a=in, b=in, out=out);  
}
```

Code description

The input **in** is connected to both inputs of the NAND gate. When $\text{in} = 0$, $\text{NAND}(0,0) = 1$. When $\text{in} = 1$, $\text{NAND}(1,1) = 0$. Therefore the output is the logical inverse of the input.

Truth Table

Input A	Output Y = NOT A
0	1
1	0

3. AND Gate Using NAND Gates

An AND gate produces 1 only when both inputs are 1. NAND first produces the inverse of AND, so a second NAND gate is used as an inverter.

Boolean expression: $Y = A \text{ AND } B$

NAND implementation: $t = A \text{ NAND } B$; $Y = t \text{ NAND } t$

HDL Code

```
CHIP And {  
  IN a, b;  
  OUT out;  
  
  PARTS:  
    Nand(a=a, b=b, out=t);  
    Nand(a=t, b=t, out=out);  
}
```

Code description

The first NAND gate generates $t = \text{NOT}(A \text{ AND } B)$. The second NAND receives t on both inputs, effectively inverting t. Hence the final output becomes $A \text{ AND } B$.

Truth Table

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

4. OR Gate Using NAND Gates

The OR function can be implemented using NAND gates through De Morgan's law. Each input is first inverted, and the inverted inputs are Nanded.

Boolean expression: $Y = A \text{ OR } B$

NAND implementation: $A' = A \text{ NAND } A$; $B' = B \text{ NAND } B$; $Y = A' \text{ NAND } B'$

HDL Code

```
CHIP Or {  
  IN a, b;  
  OUT out;  
  
  PARTS:  
    Nand(a=a, b=a, out=na);  
    Nand(a=b, b=b, out=nb);  
    Nand(a=na, b=nb, out=out);  
}
```

Code description

The first two NAND gates work as NOT gates and generate **na = NOT A** and **nb = NOT B**. The final NAND gate computes NOT(na AND nb), which is equal to A OR B by De Morgan's law.

Truth Table

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

5. XOR Gate Using NAND Gates

An XOR gate produces 1 when the two inputs are different. A standard four-NAND construction can implement XOR without using any other gate type.

Boolean expression: $Y = A \text{ XOR } B$

Intermediate equations:

```
t1 = A NAND B
t2 = A NAND t1
t3 = B NAND t1
Y = t2 NAND t3
```

HDL Code

```
CHIP Xor {
  IN a, b;
  OUT out;

  PARTS:
    Nand(a=a, b=b, out=t1);
    Nand(a=a, b=t1, out=t2);
    Nand(a=b, b=t1, out=t3);
    Nand(a=t2, b=t3, out=out);
}
```

Code description

The first NAND creates a shared intermediate signal. The next two NAND gates combine each original input with that signal. The final NAND combines these two intermediate results to produce XOR.

Truth Table

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

6. 2-Way Multiplexer (MUX) Using NAND Gates

A 2-way multiplexer selects one of two data inputs based on a select signal. If sel = 0, input a is selected. If sel = 1, input b is selected.

Boolean expression: $out = (a \text{ AND NOT } sel) \text{ OR } (b \text{ AND } sel)$

NAND-based construction:

```
nSel = sel NAND sel
na = a NAND nSel
nb = b NAND sel
out = na NAND nb
```

HDL Code

```
CHIP Mux {
  IN a, b, sel;
  OUT out;

  PARTS:
    Nand(a=sel, b=sel, out=nSel);
    Nand(a=a, b=nSel, out=na);
    Nand(a=b, b=sel, out=nb);
    Nand(a=na, b=nb, out=out);
}
```

Code description

The first NAND creates NOT sel. The next two NAND gates create the two possible selection paths. The final NAND combines the paths. Only the selected input contributes to the output.

Truth Table

sel	a	b	out
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

7. 2-Way Demultiplexer (DMUX) Using NAND Gates

A 2-way demultiplexer takes one input and routes it to one of two outputs. When sel = 0, the input is routed to a; when sel = 1, the input is routed to b.

Boolean expressions: $a = \text{in AND NOT sel}$; $b = \text{in AND sel}$

NAND-based construction:

```
nSel = sel NAND sel
ta = in NAND nSel
tb = in NAND sel
a = ta NAND ta
b = tb NAND tb
```

HDL Code

```
CHIP DMux {
  IN in, sel;
  OUT a, b;

  PARTS:
    Nand(a=sel, b=sel, out=nSel);
    Nand(a=in, b=nSel, out=ta);
    Nand(a=in, b=sel, out=tb);
    Nand(a=ta, b=ta, out=a);
    Nand(a=tb, b=tb, out=b);
}
```

Code description

The select signal is inverted using a NAND gate. Two NAND gates generate the active-low versions of the two output paths, and two more NAND gates invert those results to produce the required outputs.

Truth Table

in	sel	a	b
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

8. Complete Gate Summary and Verification

The following summary shows how every required component can be constructed exclusively from NAND gates.

Component	No. of NAND Gates	Main Idea
NOT	1	Same signal on both NAND inputs
AND	2	NAND followed by NAND-as-NOT
OR	3	Invert both inputs, then NAND
XOR	4	Four-NAND XOR structure
2-Way MUX	4	Select/invert and combine two paths
2-Way DMUX	5	Select/invert and create two output paths

Verification procedure

1. Create the HDL files using the code in this report.
2. Run each chip through an HDL simulator such as the Nand2Tetris Hardware Simulator.
3. Apply every possible input combination listed in the truth table.
4. Compare the simulator's output with the expected truth-table output.
5. If all combinations match, the NAND-only implementation is logically verified.

Important HDL note

The NAND primitive is assumed to be available as **Nand(a=..., b=..., out=...)**. Intermediate signal names such as t, na, nb, nsel, ta and tb are internal wires used to connect the NAND gates.

9. Conclusion

This project demonstrates the universality of the NAND gate by implementing six important digital components using NAND gates alone: NOT, AND, OR, XOR, 2-way MUX and 2-way DMUX.

The HDL descriptions show how a hardware design can be expressed structurally by connecting primitive NAND gates. The truth tables provide a direct method for verifying that each implementation performs the intended Boolean operation.

Key learning outcomes

- Understanding NAND as a universal gate.
- Converting Boolean expressions into NAND-only circuits.
- Writing structural HDL code using primitive gates.
- Understanding selection and routing through MUX and DMUX circuits.
- Using truth tables to verify digital logic implementations.

Project Checklist

Task	Status
NOT gate implemented using NAND	Completed
AND gate implemented using NAND	Completed
OR gate implemented using NAND	Completed
XOR gate implemented using NAND	Completed
2-way MUX implemented using NAND	Completed
2-way DMUX implemented using NAND	Completed
HDL code included	Completed
Code descriptions included	Completed
Truth tables included	Completed

End of Project Report